Mini-projet DevOps — Monitoring d'une API avec Prometheus et Grafana

> **Note** : Ce mini-projet est disponible en deux variantes au choix : **Node.js** et **Python (FastAPI)**. Les parties Infrastructure (Docker Compose, Prometheus, Grafana) sont **identiques** dans les deux cas. Seul le dossier `app/` change selon le langage choisi.

Objectifs du mini-projet

À la fin de ce mini-projet, vous devrez être capables de :

- Instrumenter une API avec les **4 types de métriques Prometheus** (Counter, Gauge, Histogram, Summary)
- Lancer via Docker Compose une stack **app + Prometheus + Grafana**
- Vérifier dans Prometheus que les métriques sont collectées et écrire des **requêtes PromQL utiles**
- Construire un **dashboard Grafana** complet avec alertes visuelles
- Appliquer les **bonnes pratiques** de nommage, de labeling et de sécurité

Structure du projet

```
monitoring-workshop/
│
├── app-node/               ← variante Node.js
│   ├── app.js
│   ├── package.json
│   └── Dockerfile
│
├── app-python/             ← variante Python
│   ├── main.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── prometheus/
│   ├── prometheus.yml
│   └── alerts.yml          ← règles d'alerte
│
```

```
├── .env.example
├── .gitignore
└── docker-compose.yml
```

Partie 0 — Fichiers communs (toutes variantes)

`.gitignore`

```
.env
node_modules/
__pycache__/
*.pyc
```

`.env.example`

```
GRAFANA_ADMIN_PASSWORD=changeme_atelier
```

> Copiez ce fichier en `.env` et modifiez le mot de passe avant de lancer la stack.

```
cp .env.example .env
```

Partie A — Application instrumentée

Choisissez **une seule** des deux variantes.

Variante 1 — Node.js

A1. Dépendances

```
mkdir -p monitoring-workshop/app-node
cd monitoring-workshop/app-node
npm init -y
npm install express prom-client
```

A2. Fichier `app.js`

```
const express = require('express');
const client  = require('prom-client');

const app  = express();
const PORT = 8080;

// ─── Registre dédié (bonne pratique : ne pas utiliser le registre global)
───
const register = new client.Registry();

// ─── Métriques système par défaut (CPU, mémoire, event loop lag, GC…)
──────
client.collectDefaultMetrics({ register, prefix: 'nodejs_' });

// ─── 1. COUNTER — nombre total de requêtes HTTP ──────────────────────────
// Convention : nom en snake_case, suffixe _total pour les counters.
const httpRequestsTotal = new client.Counter({
  name:       'http_requests_total',
  help:       'Nombre total de requêtes HTTP reçues',
  labelNames: ['method', 'route', 'status_code'],
  registers:  [register],
});

// ─── 2. HISTOGRAM — durée des requêtes HTTP ──────────────────────────────
// Les buckets doivent couvrir la plage de latences attendues.
// Convention : suffixe _seconds pour les durées.
const httpRequestDuration = new client.Histogram({
  name:       'http_request_duration_seconds',
  help:       'Durée des requêtes HTTP en secondes',
  labelNames: ['method', 'route', 'status_code'],
  buckets:    [0.01, 0.05, 0.1, 0.3, 0.5, 1, 2, 5],
  registers:  [register],
});

// ─── 3. GAUGE — requêtes en cours ────────────────────────────────────────
// Un gauge monte et descend : parfait pour une valeur instantanée.
const httpRequestsInProgress = new client.Gauge({
  name:       'http_requests_in_progress',
  help:       'Nombre de requêtes HTTP actuellement en cours de traitement',
  labelNames: ['method'],
  registers:  [register],
});

// ─── 4. GAUGE — disponibilité de l'application ───────────────────────────
const appUp = new client.Gauge({
  name:      'app_up',
```

```
  help:      '1 si l\'application est opérationnelle, 0 sinon',
  registers: [register],
});
appUp.set(1);

// ─── Middleware d'instrumentation ──────────────────────────────────────
app.use((req, res, next) => {
  // Ignorer /metrics pour ne pas polluer les données
  if (req.path === '/metrics') return next();

  const end = httpRequestDuration.startTimer();
  httpRequestsInProgress.inc({ method: req.method });

  res.on('finish', () => {
    const labels = {
      method:      req.method,
      route:       req.route ? req.route.path : req.path,
      status_code: res.statusCode,
    };
    httpRequestsTotal.inc(labels);
    end(labels);
    httpRequestsInProgress.dec({ method: req.method });
  });

  next();
});

// ─── Routes ────────────────────────────────────────────────────────────
app.get('/', (_req, res) => {
  res.json({ message: 'Hello Monitoring', timestamp: new Date().toISOString()
});
});

// Simule une latence variable (utile pour observer l'histogramme)
app.get('/slow', async (_req, res) => {
  const delay = Math.random() * 2000;
  await new Promise(r => setTimeout(r, delay));
  res.json({ message: 'Réponse lente', delay_ms: Math.round(delay) });
});

// Simule des erreurs 500
app.get('/error', (_req, res) => {
  res.status(500).json({ error: 'Erreur simulée intentionnellement' });
});

// Health check (utilisé par Docker)
```

```
app.get('/health', (_req, res) => {
  res.json({ status: 'ok' });
});

// Endpoint Prometheus — toujours en dernier, jamais instrumenté
app.get('/metrics', async (_req, res) => {
  res.set('Content-Type', register.contentType);
  res.end(await register.metrics());
});

app.listen(PORT, () => console.log(`App Node.js sur le port ${PORT}`));
```

A3. `package.json`

```
{
  "name": "monitoring-app",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": { "start": "node app.js" },
  "dependencies": {
    "express": "^4.18.2",
    "prom-client": "^15.1.0"
  }
}
```

A4. `Dockerfile` (Node.js)

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
# npm ci garantit un build reproductible (utilise package-lock.json)
RUN npm ci --omit=dev

COPY . .

EXPOSE 8080

HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD node -e "require('http').get('http://localhost:8080/health', r =>
process.exit(r.statusCode===200?0:1)).on('error',()=>process.exit(1))"

CMD ["node", "app.js"]
```

Variante 2 — Python (FastAPI)

A1. Dépendances

```
mkdir -p monitoring-workshop/app-python
```

A2. `requirements.txt`

```
fastapi==0.110.0
uvicorn==0.29.0
prometheus-client==0.20.0
```

A3. Fichier `main.py`

```
import time
import random
from fastapi import FastAPI, Response, Request
from prometheus_client import (
    Counter, Gauge, Histogram,
    CollectorRegistry, generate_latest, CONTENT_TYPE_LATEST,
    multiprocess, values,
)

app = FastAPI()

# ─── Registre dédié ──────────────────────────────────────────────────
registry = CollectorRegistry()

# ─── 1. COUNTER — nombre total de requêtes HTTP ──────────────────────
http_requests_total = Counter(
    "http_requests_total",
    "Nombre total de requêtes HTTP reçues",
    ["method", "route", "status_code"],
    registry=registry,
)

# ─── 2. HISTOGRAM — durée des requêtes HTTP ──────────────────────────
http_request_duration_seconds = Histogram(
    "http_request_duration_seconds",
    "Durée des requêtes HTTP en secondes",
    ["method", "route", "status_code"],
    buckets=[0.01, 0.05, 0.1, 0.3, 0.5, 1, 2, 5],
```

```
    registry=registry,
)

# ─── 3. GAUGE — requêtes en cours ──────────────────────────────────────
http_requests_in_progress = Gauge(
    "http_requests_in_progress",
    "Nombre de requêtes HTTP actuellement en cours",
    ["method"],
    registry=registry,
)

# ─── 4. GAUGE — disponibilité ──────────────────────────────────────────
app_up = Gauge("app_up", "1 si l'application est opérationnelle",
registry=registry)
app_up.set(1)


# ─── Middleware d'instrumentation ──────────────────────────────────────
@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    if request.url.path == "/metrics":
        return await call_next(request)

    method = request.method
    route  = request.url.path

    http_requests_in_progress.labels(method=method).inc()
    start = time.perf_counter()

    response = await call_next(request)

    duration    = time.perf_counter() - start
    status_code = str(response.status_code)
    labels      = {"method": method, "route": route, "status_code":
status_code}

    http_requests_total.labels(**labels).inc()
    http_request_duration_seconds.labels(**labels).observe(duration)
    http_requests_in_progress.labels(method=method).dec()

    return response


# ─── Routes ────────────────────────────────────────────────────────────
@app.get("/")
def root():
```

```python
    return {"message": "Hello Monitoring", "lang": "Python"}


@app.get("/slow")
async def slow():
    import asyncio
    delay = random.uniform(0.1, 2.0)
    await asyncio.sleep(delay)
    return {"message": "Réponse lente", "delay_ms": round(delay * 1000)}


@app.get("/error")
def error():
    from fastapi import HTTPException
    raise HTTPException(status_code=500, detail="Erreur simulée
intentionnellement")


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/metrics")
def metrics():
    data = generate_latest(registry)
    return Response(content=data, media_type=CONTENT_TYPE_LATEST)
```

A4. `Dockerfile` (Python)

```dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8080/health')"
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Partie B — Configuration Prometheus

`prometheus/prometheus.yml`

```yaml
global:
  scrape_interval:     15s   # Valeur standard en production
  evaluation_interval: 15s   # Fréquence d'évaluation des règles d'alerte
  # En atelier uniquement, vous pouvez descendre à 5s pour un feedback plus
rapide

# Chargement des règles d'alerte
rule_files:
  - alerts.yml

scrape_configs:
  - job_name: app
    static_configs:
      - targets: ['app:8080']
    # Bonne pratique : ajouter des métadonnées pour filtrer dans Grafana
    relabel_configs:
      - source_labels: [__address__]
        target_label: instance

  # Prometheus se monitore lui-même
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']
```

`prometheus/alerts.yml`

```yaml
groups:
  - name: app_alerts
    rules:

      # Alerte si l'application est DOWN
      - alert: AppDown
        expr: up{job="app"} == 0
        for: 30s
        labels:
```

```
          severity: critical
        annotations:
          summary: "L'application est inaccessible"
          description: "Le job {{ $labels.job }} sur {{ $labels.instance }}
est DOWN depuis plus de 30 secondes."

      # Alerte si le taux d'erreur dépasse 10%
      - alert: HighErrorRate
        expr: |
          rate(http_requests_total{status_code=~"5.."}[5m])
          /
          rate(http_requests_total[5m]) > 0.10
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "Taux d'erreur HTTP élevé (> 10%)"
          description: "Le taux d'erreur est de {{ humanizePercentage $value
}} sur {{ $labels.instance }}."

      # Alerte si la latence p99 dépasse 1 seconde
      - alert: HighLatency
        expr: |
          histogram_quantile(0.99,
            rate(http_request_duration_seconds_bucket[5m])
          ) > 1
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "Latence p99 élevée (> 1s)"
          description: "La latence au 99e percentile est de {{
humanizeDuration $value }} sur {{ $labels.instance }}."
```

Partie C — Docker Compose

`docker-compose.yml`

```
services:

  # ── Application (choisir une seule variante) ──────────────────────────
  app:
    build: ./app-node        # Remplacer par ./app-python si variante Python
```

```yaml
    container_name: monitoring_app
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/health || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 15s

  # ── Prometheus ──────────────────────────────────────────────────────
  prometheus:
    image: prom/prometheus:v2.51.0   # Version fixée : reproductibilité
garantie
    container_name: monitoring_prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/alerts.yml:/etc/prometheus/alerts.yml:ro
      - prometheus_data:/prometheus
    ports:
      - "9090:9090"
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.retention.time=15d'   # Rétention des données : 15
jours
    depends_on:
      app:
        condition: service_healthy   # Attend que l'app soit prête avant de
scraper

  # ── Grafana ─────────────────────────────────────────────────────────
  grafana:
    image: grafana/grafana:10.3.3   # Version fixée
    container_name: monitoring_grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}   # Depuis .env
      - GF_USERS_ALLOW_SIGN_UP=false
    volumes:
      - grafana_data:/var/lib/grafana   # Persiste les dashboards entre les
redémarrages

volumes:
  prometheus_data:   # Persiste les métriques Prometheus
  grafana_data:      # Persiste les dashboards Grafana
```

Partie D — Lancement et vérifications

Démarrer la stack

```
cd monitoring-workshop

# Copier et remplir les variables d'environnement
cp .env.example .env

docker compose up --build -d
```

Vérifications de base

URL	Résultat attendu
`http://localhost:8080/`	`{"message": "Hello Monitoring", ...}`
`http://localhost:8080/slow`	Réponse après 0,1 à 2 secondes
`http://localhost:8080/error`	Code HTTP 500
`http://localhost:8080/health`	`{"status": "ok"}`
`http://localhost:8080/metrics`	Métriques au format texte Prometheus
`http://localhost:9090/`	Interface Prometheus
`http://localhost:3000/`	Interface Grafana

Partie E — Prometheus : explorer les métriques

Vérifier que la target est active

Aller dans **Status** → **Targets** : la target `app` doit être en `UP`.

Générer du trafic

```
# Générer des requêtes normales
for i in $(seq 1 20); do curl -s http://localhost:8080/ > /dev/null; done

# Générer des requêtes lentes
for i in $(seq 1 5); do curl -s http://localhost:8080/slow > /dev/null; done
```

```
# Générer des erreurs
for i in $(seq 1 5); do curl -s http://localhost:8080/error > /dev/null; done
```

Requêtes PromQL à tester

Testez chacune dans l'onglet **Graph** de Prometheus.

Volume de trafic — taux de requêtes par seconde :

```
rate(http_requests_total[1m])
```

Taux d'erreur en % (rapport 5xx / total) :

```
rate(http_requests_total{status_code=~"5.."}[1m])
/
rate(http_requests_total[1m]) * 100
```

Latence médiane (p50) :

```
histogram_quantile(0.50, rate(http_request_duration_seconds_bucket[5m]))
```

Latence au 99e percentile (p99) — les cas les plus lents :

```
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

Nombre de requêtes en cours à l'instant T :

```
http_requests_in_progress
```

Disponibilité de l'application :

```
up{job="app"}
```

Partie F — Grafana : construire le dashboard

Configurer la data source Prometheus

1. Aller dans **Connections** → **Data sources** → **Add data source**

2. Choisir **Prometheus**
3. URL : `http://prometheus:9090`
4. Cliquer **Save & test** — vérifier que la connexion est verte

Panels à créer

Créer un nouveau dashboard et ajouter les panels suivants :

Panel 1 — Taux de requêtes HTTP (time series)

```
rate(http_requests_total[1m])
```

Configurer la légende : `{{method}} {{route}} {{status_code}}`

Panel 2 — Taux d'erreur en % (stat ou gauge)

```
rate(http_requests_total{status_code=~"5.."}[1m])
/
rate(http_requests_total[1m]) * 100
```

Ajouter un seuil visuel : vert sous 5%, orange entre 5% et 10%, rouge au-dessus.

Panel 3 — Latences p50 / p95 / p99 (time series)

```
histogram_quantile(0.50, rate(http_request_duration_seconds_bucket[5m]))
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

Panel 4 — Disponibilité (stat)

```
up{job="app"}
```

Seuils : 1 = vert (UP), 0 = rouge (DOWN).

Panel 5 — Requêtes en cours (gauge)

```
http_requests_in_progress
```

Démonstration "arrêt de l'application"

Ceci est **la manipulation la plus importante du TP**. Elle justifie à elle seule l'intérêt du monitoring.

```
# Arrêter l'application
docker stop monitoring_app

# Observer dans Prometheus (Status → Targets) : la target passe en DOWN
# Observer dans Grafana : le panel "Disponibilité" passe au rouge
# Après ~30s, Prometheus déclenche l'alerte AppDown

# Redémarrer l'application
docker start monitoring_app

# Observer la récupération sur le dashboard
```

Partie G — Bonnes pratiques à retenir

Nommage des métriques

Règle	Correct	Incorrect
Snake case	`http_requests_total`	`httpRequestsTotal`
Suffixe de type pour les counters	`requests_total`	`requests`
Unité dans le nom	`duration_seconds`	`duration`
Préfixe applicatif	`myapp_errors_total`	`errors_total`

Les 4 types de métriques

Type	Quand l'utiliser	Exemple
Counter	Valeur qui ne fait que croître	Nombre de requêtes, d'erreurs
Gauge	Valeur qui monte et descend	Connexions actives, usage mémoire
Histogram	Distribution de valeurs, percentiles	Durée des requêtes, taille des réponses
Summary	Percentiles pré-calculés côté app	Rarement préféré à l'Histogram

Ce qu'il ne faut jamais faire

- **Ne pas instrumenter** `/metrics` : cela crée une récursion infinie dans les métriques

- **Ne pas utiliser trop de labels** : chaque combinaison de labels crée une série temporelle distincte — éviter les labels à cardinalité élevée (ex: ID utilisateur, adresse IP)
- **Ne pas exposer `/metrics` publiquement** en production : cette route révèle des informations système sensibles

---

Livrables attendus

Le mini-projet est considéré comme terminé lorsque les conditions suivantes sont remplies.

1. Stack fonctionnelle

- L'application répond sur `/`, `/slow`, `/error`, `/health` et `/metrics`
- Prometheus est accessible et la target `app` est en `UP`
- Grafana est accessible avec une data source Prometheus fonctionnelle

2. Dashboard complet

- Les 5 panels sont présents et affichent des données non nulles
- Le panel "Taux d'erreur" change de couleur après un appel à `/error`
- La démonstration "arrêt de l'application" a été réalisée et observée

3. Analyse écrite (15-20 lignes)

Répondre aux questions suivantes dans un fichier `ANALYSE.md` :

1. Quelle est la différence entre un **Counter** et un **Gauge** ? Donnez un exemple pour chacun.
2. Pourquoi utilise-t-on un **Histogram** plutôt qu'un Counter pour mesurer la latence ?
3. Qu'est-ce que le **p99** et pourquoi est-il plus révélateur que la moyenne ?
4. Que se passe-t-il dans Prometheus et Grafana quand vous arrêtez l'application ?
5. Pourquoi ne faut-il **pas** utiliser l'ID utilisateur comme label Prometheus ?

Grille d'évaluation

Critère	Points
Stack Docker : les 3 services démarrent correctement	3
Les 4 types de métriques sont présents dans `/metrics`	4
Les 5 panels Grafana sont configurés correctement	5

Critère	Points
La démonstration "app DOWN" a été réalisée	2
Les requêtes PromQL avancées (p99, taux d'erreur) fonctionnent	3
Analyse écrite : réponses complètes et correctes	3
Total	**20**

Pour aller plus loin (facultatif)

- Ajouter **Alertmanager** pour recevoir les alertes par email ou Slack
- Configurer un **panel de type Heatmap** pour visualiser la distribution complète de l'histogramme de latence
- Ajouter des métriques **métier** personnalisées (ex: nombre de commandes créées, panier moyen)
- Explorer **Loki + Promtail** pour agréger les logs applicatifs dans Grafana